

End-to-End MERN Application Architecture & Best Practices

Name: Sibulangcao, Zyrus Emmanuel Q.

Student ID: 24-0043-286

MERN Lab Project — Challenges & Reflections

Building the MERN lab project came with a fair share of hurdles, particularly around project setup, MongoDB connectivity, React configuration, and API testing.

The first issue I ran into was a MongoDB connection error caused by deprecated options — specifically `useNewUrlParser` and `useUnifiedTopology`. Since the version of Mongoose I had installed no longer required these, I simply removed them from `db.js`, which cleared the error right away.

On the frontend side, the React app refused to start, throwing a "Could not find a required file: `index.js`" error. It turned out that `src/index.js` was missing entirely. Recreating the file and restoring the proper project structure got things running again.

API testing in Postman surfaced another problem: a "Cast to ObjectId failed" error when trying to create a task document. The culprit was a placeholder string being used in place of a real MongoDB ObjectId. I worked around this by first creating a user document, then using its generated `_id` as the `userId` when creating tasks.

Beyond specific bugs, I also had to think carefully about how to organize the project as a whole — and more broadly, how the backend and frontend actually talk to each other in a MERN app. Breaking the codebase into dedicated folders (models, routes, config, and components) made everything significantly cleaner and easier to navigate.

By the end of the lab, I had hands-on experience with:

- Connecting MongoDB through Mongoose
- Building REST API routes with Express
- Performing CRUD operations
- Working with React components and state
- Debugging common MERN stack errors
- Improving code organization and readability

All in all, the lab gave me a much clearer picture of how the MERN stack fits together — and how to methodically work through the problems that come with it.

SCREENSHOT OF FRONTEND

